

Process Scheduling

Process scheduling is a fundamental concept in operating systems (OS) that manages how multiple programs (called processes) share limited computer resources, like the CPU (Central Processing Unit), memory, and input/output (I/O) devices. Imagine your computer as a busy kitchen where multiple chefs (processes) need to use the stove (CPU) and ingredients (resources). Without scheduling, everything would be chaotic—chefs fighting over the stove, leading to inefficiency or crashes. Scheduling ensures fair, efficient, and organized use of resources.

This section covers everything from basics (what a process is) to advanced topics (types of schedulers, queues, and states). We'll build step by step so even a beginner can follow.

Why Process Scheduling Is Required

At the most basic level, a **process** is a program in execution. For example, when you open a web browser like Chrome, it becomes a process. Modern computers run many processes at once (multitasking), but resources like the CPU are limited—most CPUs can only execute one process at a time per core (though multi-core CPUs allow parallelism).

Basics: The Need for Scheduling

- **Resource Limitations:** Computers have finite resources. The CPU can't run everything simultaneously, memory (RAM) has limits, and I/O devices (like hard drives or printers) are slow compared to the CPU.
- **Multitasking Illusion:** Your OS makes it seem like multiple apps run at once (e.g., listening to music while browsing), but it's actually switching rapidly between processes.
- **Key Reasons for Scheduling:**
 - **Resource Utilization:** Maximizes use of resources. Without scheduling, the CPU might sit idle while a process waits for I/O (e.g., loading a file), wasting time. Scheduling switches to another process during waits, keeping the CPU busy (aiming for 100% utilization).

- **Fairness**: Ensures no single process hogs resources (starvation prevention). For example, a background update shouldn't freeze your game.
- **Efficiency**: Reduces waiting times, improves throughput (processes completed per unit time), and minimizes response time (how quickly a process reacts to input).
- **Prioritization**: Some processes are more important (e.g., system tasks over user apps), so scheduling assigns priorities.
- **System Stability**: Prevents overload; too many processes could crash the system due to memory exhaustion.

Advanced: Metrics and Goals

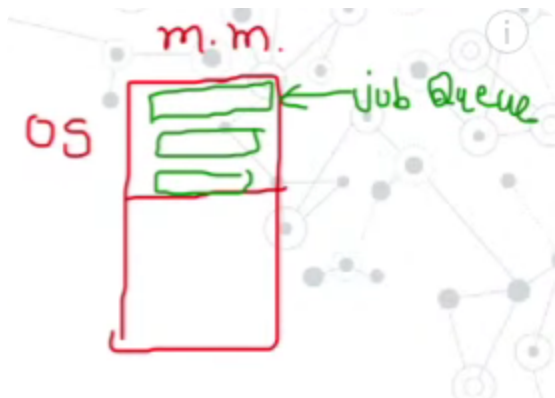
- Scheduling algorithms aim to optimize:
 - **CPU Utilization**: Percentage of time CPU is busy (high is good).
 - **Throughput**: Number of processes finished per hour (high is good).
 - **Turnaround Time**: Total time from process submission to completion (low is good).
 - **Waiting Time**: Time a process spends waiting in queues (low is good).
 - **Response Time**: Time from request to first response (low for interactive systems like games).
- In real-time systems (e.g., medical devices), scheduling ensures deadlines are met to avoid failures.
- Trade-offs: Optimizing one metric might worsen another (e.g., high throughput might increase waiting time).

Because Resource Utilization: This is the core driver—scheduling prevents idle resources, boosting overall system performance.

Scheduling Queues

Scheduling queues are data structures (like lists or linked lists) used by the OS to organize processes based on their current state. Each process has a **Process Control Block (PCB)**—a data structure storing info like process ID, state, priority, CPU registers, and memory allocation. Queues hold pointers to these PCBs, not the full processes, for efficiency.

Think of queues as airport lines: one for check-in (new processes), one for security (ready to fly), and one for boarding gates (waiting for specific flights/devices).



Basics: What Are Scheduling Queues?

- Queues help the OS track and manage processes in different states:
 - **New**: Process just created.
 - **Ready**: Loaded in memory, waiting for CPU.
 - **Running**: Currently executing on CPU.
 - **Waiting/Blocked**: Paused for I/O or events.
 - **Terminated**: Finished or killed.
- Queues use FIFO (First-In-First-Out) or priority-based ordering.

Main Types of Queues

- **Job Queue**:
 - **Basics**: Holds all processes in the **New** state. These are freshly created but not yet in memory.
 - **Function**: Processes wait here to be selected for memory allocation.
 - **Example**: Launching apps like Word and Excel—they enter the job queue before loading.
 - **Advanced**: In batch systems (e.g., old mainframes), this queue handles job submissions. Overloading it can cause delays in admitting processes.
- **Ready Queue**:
 - **Basics**: Holds processes in the **Ready** state, loaded in memory and eager for CPU time.

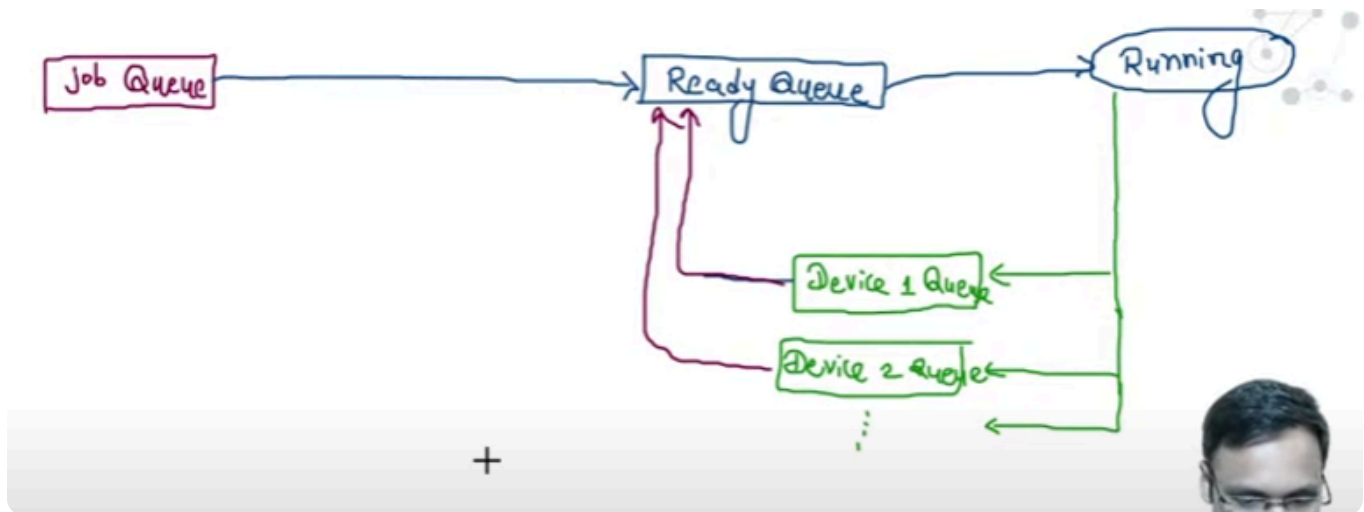
- **Function**: The CPU scheduler picks from here to run a process.
- **Example**: Word and Excel are in memory but waiting because Photoshop is running.
- **Advanced**: Can be multi-level (e.g., separate queues for high/low priority). In preemptive scheduling, a running process can be interrupted and sent back here.
- **Device Queue** (also called I/O Queue or Waiting Queue):
 - **Basics**: Holds processes in the **Waiting/Blocked** state, paused for I/O (e.g., reading a file) or devices.
 - **Function**: Each device has its own queue to manage requests without blocking the whole system.
 - **Example**: A process printing a document waits in the printer's device queue.
 - **Advanced**: Queues can prioritize I/O requests (e.g., shortest seek time first for disks). Processes here are blocked until the I/O completes, then move back to ready queue.

Does Each Device Have Its Own Device Queue?: Yes! For efficiency, the OS maintains separate queues per device (e.g., one for disk, one for network, one for printer). This prevents a slow printer from delaying disk reads. If multiple processes want the same device, they queue up specifically for it.

Process Flow Through Queues

- **Basic Flow (Non-Preemptive)**: Job Queue (New) → Ready Queue (admitted to memory) → Running State (gets CPU) → Device Queue (if I/O needed) → Ready Queue (after I/O) → Running → Terminated.
- **In Case of Preemption**: In preemptive scheduling (e.g., time-sharing), a running process can be interrupted (e.g., time slice ends) and sent back to Ready Queue. Example: Process A runs, gets preempted by higher-priority Process B, A goes to Ready Queue.
- **Advanced**: Queues hold PCBs, not full processes, to save space. In multi-queue systems, feedback loops allow processes to move between queues based on behavior (e.g., CPU-bound vs. I/O-bound).

Queues Will Hold PCB of the processes in Device Queues: Yes, all queues store pointers to PCBs. This allows quick state changes without copying large data.



Advanced: Queue Implementation

- Queues can be single-level (simple list) or multi-level (e.g., priority queues).
- In real OS like Linux, ready queues are per-CPU in multi-core systems for load balancing.
- Potential Issues: Queue starvation (low-priority processes wait forever) or thrashing (too much queue switching overhead).

Types of Schedulers

Schedulers are OS algorithms/components that decide which process gets resources and when. They handle transitions between states and queues. There are three main types, based on time scale and role.

Basics: Scheduler Overview

- Schedulers work with the dispatcher (a low-level component that switches CPU context—saving/restoring registers from PCBs).
- They classify processes as:
 - **CPU-Bound**: Spend more time computing (e.g., video rendering).
 - **I/O-Bound**: Spend more time waiting for I/O (e.g., web browsing).
- **Long-Term Scheduler (Job Scheduler)**:

- **Basics**: Operates at a high level, deciding which new processes enter the system.
- **Role**: Controls the **degree of multiprogramming** (max number of processes in memory at once, e.g., 5-10 to avoid overload).
- **Function**: Selects from Job Queue and admits to Ready Queue, allocating memory.
- **Example**: On a phone with low RAM, it might delay loading a new game if too many apps are open.
- **Frequency**: Runs infrequently (e.g., on process creation or termination).
- **Advanced**: In batch systems, it balances CPU/I/O-bound processes for optimal utilization. If degree is too high, thrashing occurs (excessive swapping).
- **Short-Term Scheduler (CPU Scheduler)**:
 - **Basics**: Decides which ready process gets the CPU next.
 - **Role**: Manages CPU allocation for fast context switches.
 - **Function**: Picks from Ready Queue and dispatches to Running state.
 - **Example**: Switches between a music player and email app every few ms.
 - **Frequency**: Very frequent (e.g., every 10-100 ms in time-sharing OS).
 - **Advanced**: Uses algorithms like FCFS (First-Come-First-Served), SJF (Shortest Job First), Priority, or Round-Robin. Preemptive vs. Non-Preemptive: Preemptive interrupts running processes; non-preemptive waits for completion.
- **Mid-Term Scheduler (Medium-Term Scheduler)**:
 - **Basics**: Handles memory management by temporarily removing processes.
 - **Role**: Reduces degree of multiprogramming when system is overloaded.
 - **Function**: Performs **swapping** (moving processes between RAM and disk).
 - **Swap Out**: Inactive process to secondary storage (frees RAM).
 - **Swap In**: Brings back when space/need arises.
 - **Example**: Swaps out a minimized browser to free RAM for a video editor.
 - **Frequency**: Occasional, triggered by memory pressure or timers.
 - **Advanced**: Swapping is also called "rolling" in some contexts. If based on priority, low-priority processes swap first. Improves throughput by allowing more active processes in RAM.

Swapping is known as rolling also, if swapping is based on priority of processes:

Yes, "rolling" is an older term for swapping. Priority-based swapping ensures critical processes stay in memory.

Where The Swapped Processes are stored?: In **Swap Space**—a dedicated partition on secondary storage (HDD/SSD) managed by the OS. It's like a temporary parking lot for processes, invisible to users.

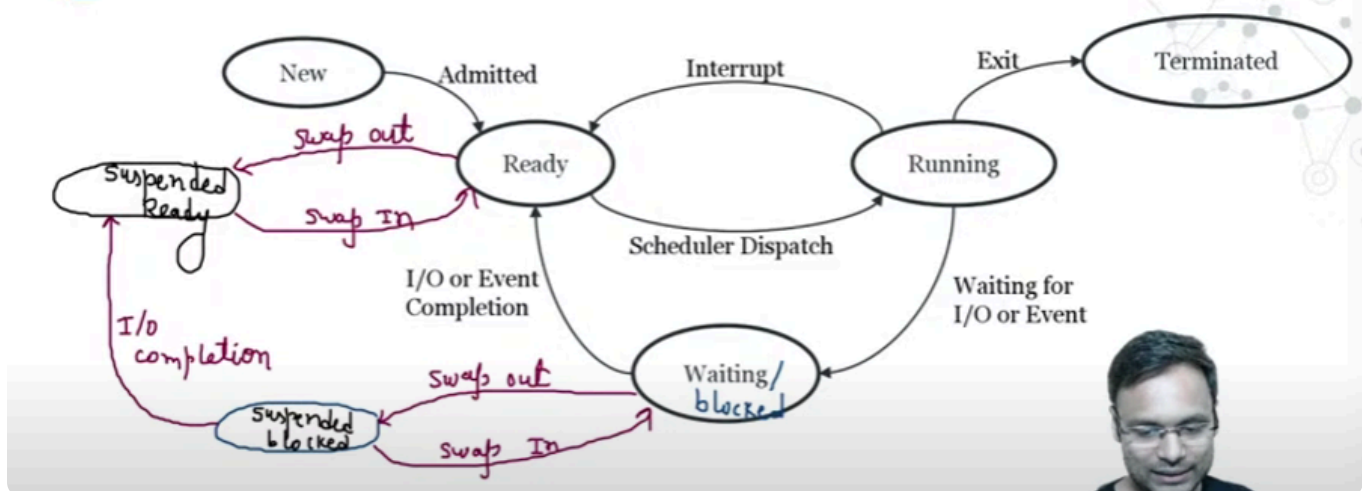
Swapped Out Process Is Called Suspended Process And It goes in Suspended State and Then sent to Waiting/Blocked State Or Ready State when unsuspended:

- **Suspended State:** A process is suspended (swapped out) when inactive or low-priority.
- **Suspended Ready State:** A sub-state where a suspended process is ready but still on disk; it needs swap-in to become truly ready.
- When unsuspended (swapped in), it goes to Ready (if no I/O wait) or Waiting (if blocked).

Process States and Diagram

- **7-State Model** (expanding the basic 5 states):
 - New → Ready → Running → Terminated (basic).
 - Plus: Waiting/Blocked, Suspended Ready (swapped but ready), Suspended Blocked (swapped and waiting).
- Transitions: E.g., Ready → Suspended Ready (swap out), Suspended Ready → Ready (swap in).

Updated Process States



Process That Are In New States Does They Have There PCB Made Up? And If Yes, Where Are They Kept?:

- **Yes**, even in New state, a PCB is created immediately upon process creation (via `fork()` in Unix-like OS).
- **Where Kept?**: In the Job Queue (as pointers). PCBs are stored in kernel memory (a protected OS area), not user space, for quick access. This allows the long-term scheduler to inspect details like priority before admitting to Ready Queue.

Advanced: Scheduler Interactions

- Long-term sets the "big picture" (admission), mid-term manages memory (swapping), short-term handles CPU (execution).
- In modern OS (e.g., Windows, Linux), these are integrated; Linux uses Completely Fair Scheduler (CFS) for short-term.
- Overhead: Frequent scheduling can cause context switch overhead (saving/restoring state), so balance is key.

CPU Scheduling

CPU scheduling is a critical function of the operating system (OS) that determines which process in the **Ready Queue** gets to use the CPU at any given time. It's managed by the **Short-Term Scheduler** (also called the CPU Scheduler). Think of it as a traffic controller at a busy intersection, deciding which car (process) moves next to keep traffic flowing smoothly. This section explains CPU scheduling from the ground up, covering its purpose, goals, and related concepts, making it clear for beginners while diving into advanced details.

Overview: What Is CPU Scheduling?

At its core, CPU scheduling decides which process from the **Ready Queue**—a list of processes loaded in memory and ready to execute—gets to run on the CPU. Since most CPUs can only execute one process at a time per core, the OS must juggle multiple processes to create the illusion of multitasking (e.g., playing music while browsing the web). The **Short-Term Scheduler** makes these decisions rapidly, often in milliseconds, to ensure efficient and fair use of the CPU.

Done By Short-Term Scheduler

- **Role:** Selects a process from the **Ready Queue** and allocates CPU time to it.
- **Function:** Works with the **dispatcher** (a low-level OS component) to perform context switches—saving the state of the current process (in its **Process Control Block (PCB)**) and loading the state of the next process.
- **Example:** When you switch from typing in a Word document to scrolling a webpage, the short-term scheduler decides which process (Word or Chrome) gets the CPU next.
- **Frequency:** Runs frequently, often every 10-100 milliseconds in time-sharing systems, or when a process blocks (e.g., waits for I/O) or terminates.

Basics: Why CPU Scheduling Is Needed

- **Limited CPU Resources:** A single-core CPU can only run one process at a time, even if many are ready.
- **Multitasking:** Users expect multiple apps to run smoothly together (e.g., Zoom, Spotify, and a game).
- **Efficiency:** Without scheduling, the CPU could sit idle while a process waits for I/O (like loading a file), wasting resources.
- **Prioritization:** Some processes (e.g., system tasks) need CPU time sooner than others (e.g., background updates).

Advanced: Context of CPU Scheduling

- CPU scheduling operates within the broader process scheduling framework, alongside:
 - **Long-Term Scheduler:** Decides which new processes enter memory (controls degree of multiprogramming).
 - **Mid-Term Scheduler:** Manages memory by swapping processes to/from disk.
- The short-term scheduler focuses solely on CPU allocation for processes already in memory (Ready Queue).
- In multi-core systems, each core has its own scheduler or shares a global ready queue for load balancing.

Goals of CPU Scheduling

The short-term scheduler aims to optimize system performance while balancing competing objectives. Here are the main goals, explained for beginners and with advanced insights:

Minimize Waiting Time and Turnaround Time

- **Waiting Time:**
 - **Definition:** The total time a process spends in the **Ready Queue** waiting for the CPU.
 - **For Beginners:** Imagine waiting in line at a coffee shop. The longer you wait, the more frustrated you get. Similarly, processes wait in the Ready Queue until the scheduler picks them.
 - **Example:** If Spotify is in the Ready Queue while a game uses the CPU, its waiting time is the delay before it gets CPU time to play music.
 - **Advanced:** Waiting time excludes time spent in I/O (Waiting/Blocked state) or execution (Running state). It's a key metric for interactive systems where quick response matters (e.g., clicking a button in an app).
 - **Goal:** Keep waiting time low to make the system feel responsive.
- **Turnaround Time:**
 - **Definition:** The total time from when a process is created (enters the Job Queue) to when it completes (Terminated state). It includes:
 - Waiting time in the Ready Queue.
 - Execution time on the CPU.
 - Time spent in I/O or Waiting/Blocked state.
 - **For Beginners:** Think of turnaround time as the total time to complete a task, from starting a download to having the file ready. It's the "end-to-end" time for a process.
 - **Example:** For Spotify, turnaround time includes the time to launch, load a song (I/O), and play it (CPU execution).
 - **Advanced:** Turnaround time = Waiting Time + Execution Time + I/O Time. Minimizing it improves system efficiency, especially in batch systems where many processes run sequentially.
 - **Goal:** Reduce turnaround time to complete processes faster.

Maximize CPU Utilization (Throughput)

- **Throughput:**
 - **Definition:** The number of processes completed per unit of time (e.g., processes per second).
 - **For Beginners:** Imagine a factory producing widgets. Throughput is how many widgets get made in an hour. For the CPU, it's how many processes finish in a given time.
 - **Example:** If the OS completes 10 apps (e.g., opening Chrome, saving a file in Word) in a minute, that's the throughput.
 - **Advanced:** High throughput requires keeping the CPU busy, avoiding idle time when processes wait for I/O or other events. Scheduling algorithms like Shortest Job First (SJF) maximize throughput by prioritizing quick tasks.
 - **Goal:** Maximize throughput to get more work done.
- **CPU Utilization:**
 - **Definition:** The percentage of time the CPU is actively executing processes (vs. being idle).
 - **For Beginners:** If the CPU is a chef, utilization is how much time they spend cooking vs. standing idle. The OS wants the CPU "cooking" as much as possible.
 - **Example:** If Spotify waits for a song to buffer (I/O), the CPU can switch to Chrome to stay busy, increasing utilization.
 - **Advanced:** Ideal utilization is 100%, but I/O-bound processes (e.g., waiting for network data) can lower it. Scheduling balances CPU-bound (computation-heavy) and I/O-bound processes to keep the CPU active.
 - **Goal:** Achieve high CPU utilization (close to 100%) for efficiency.

Fairness / Non-Biased

- **Definition:** Ensuring all processes get reasonable CPU time, preventing any single process from monopolizing the CPU (starvation).
- **For Beginners:** Imagine a teacher giving equal time to students to speak in class. The OS ensures no app hogs the CPU, so all apps (e.g., a game and a background update) work smoothly.
- **Example:** A game shouldn't freeze a background virus scan; the scheduler gives both CPU time fairly.

- **Advanced**: Fairness is achieved through algorithms like Round-Robin (time-sharing) or priority-based scheduling with aging (increasing priority for long-waiting processes). Starvation occurs when low-priority processes are perpetually ignored, so modern OS like Linux use techniques like Completely Fair Scheduler (CFS) to balance fairness and performance.
- **Goal**: Prevent starvation and ensure equitable CPU access.

Advanced: Trade-Offs and Metrics

- **Balancing Goals**: Optimizing one goal can hurt another. For example:
 - Minimizing waiting time (favoring short jobs) may reduce throughput if long jobs are delayed.
 - Maximizing throughput (prioritizing quick tasks) may increase turnaround time for complex tasks.
- **Other Goals**:
 - **Response Time**: Time from a user request (e.g., clicking a button) to the first response. Critical for interactive systems like GUIs or games.
 - **Predictability**: In real-time systems (e.g., medical devices), scheduling ensures deadlines are met consistently.
- **Scheduling Algorithms**: Different algorithms prioritize different goals:
 - **First-Come-First-Served (FCFS)**: Simple but can cause long waits (poor fairness).
 - **Shortest Job First (SJF)**: Minimizes waiting time but risks starvation for long jobs.
 - **Round-Robin (RR)**: Fair, good for time-sharing, but may increase turnaround time.
 - **Priority Scheduling**: Favors high-priority tasks but needs anti-starvation mechanisms.
 - **Multilevel Queue**: Separates processes (e.g., system vs. user) for tailored scheduling.

Question 1: Which Scheduler Reduces the Degree of Multiprogramming?

The **degree of multiprogramming** is the number of processes loaded in memory (RAM) at once. Reducing it means removing processes from memory to free resources, typically by swapping them to secondary storage (disk).

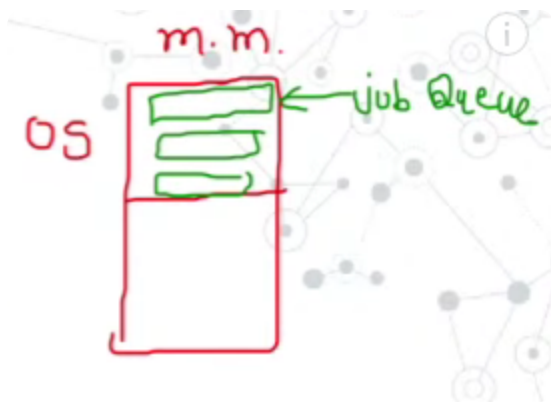
- **Options:**
 - **Short-Term Scheduler:** Manages CPU allocation, picking processes from the Ready Queue to run. It doesn't add or remove processes from memory, so it doesn't reduce multiprogramming.
 - **Long-Term Scheduler:** Controls which processes from the Job Queue enter memory (Ready Queue). By admitting fewer processes, it can limit multiprogramming but doesn't actively reduce it once processes are in memory.
 - **Mid-Term Scheduler:** Handles swapping—moving processes from RAM to disk (swap space) when memory is full or a process is inactive. This directly reduces the number of processes in memory.
 - **Long and Mid:** Combines the above.
- **Correct Answer: Mid-Term Scheduler**
 - **Explanation:** The mid-term scheduler reduces the degree of multiprogramming by swapping out processes (e.g., inactive or low-priority ones) to secondary storage. This frees up memory for other processes or prevents system overload. The long-term scheduler controls initial admission but doesn't remove processes already in memory, and the short-term scheduler only allocates CPU time.

Advanced Notes on Multiprogramming

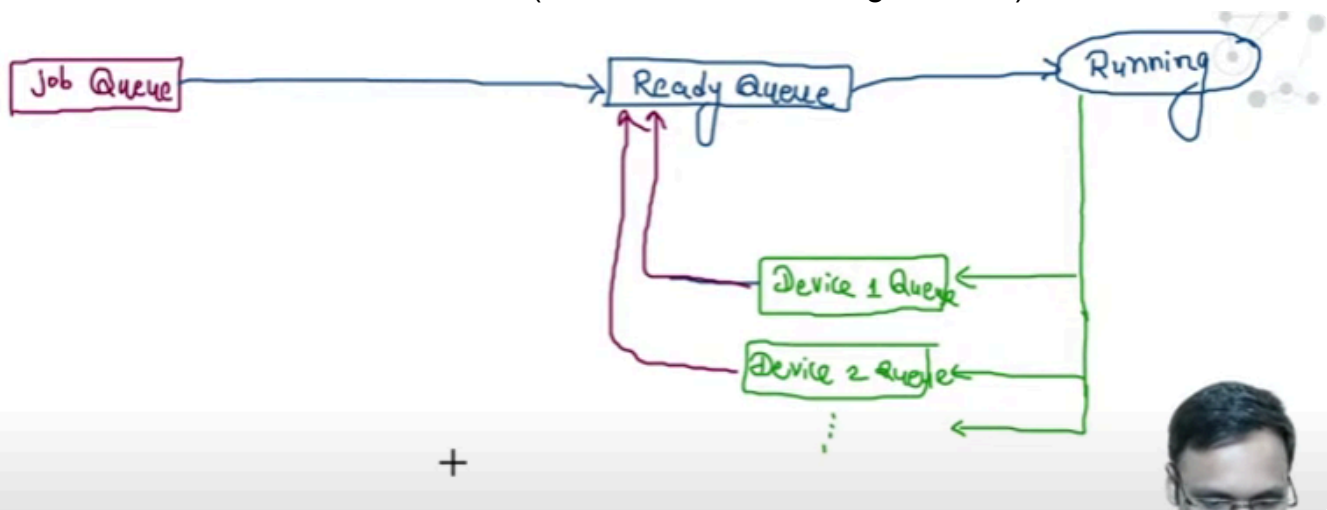
- **Why Reduce Multiprogramming?:** Too many processes in memory cause **thrashing**—excessive swapping that slows the system due to disk I/O. The mid-term scheduler prevents this by suspending processes.
- **Swapped Processes:** Stored in **swap space** (a dedicated disk partition). These are in **Suspended Ready** (ready but swapped) or **Suspended Blocked** (waiting but swapped) states.
- **Example:** If you have 10 apps open but RAM is full, the mid-term scheduler might swap out a minimized app (e.g., a paused game) to disk, reducing the degree of multiprogramming from 10 to 9.

Additional Notes

- **Process Control Block (PCB)**: Every process, even in the New state, has a PCB created upon process creation (e.g., via `fork()` in Linux). PCBs are stored in kernel memory and referenced in queues (Job, Ready, Device) for quick state management.
- **Preemption**: In preemptive scheduling (e.g., Round-Robin), the short-term scheduler can interrupt a running process (e.g., after a time slice) and move it back to the Ready Queue, allowing another process to run. This enhances fairness and responsiveness.
- **Modern OS Examples**:
 - Linux uses the **Completely Fair Scheduler (CFS)**, which dynamically adjusts process priorities based on time spent waiting and CPU usage, balancing fairness and efficiency.
 - Windows uses a priority-driven preemptive scheduler with multilevel feedback queues.
- **Challenges**:
 - **Context Switch Overhead**: Switching processes takes time (saving/restoring PCB data), so excessive scheduling reduces efficiency.
 - **Starvation**: Poor scheduling can leave low-priority processes waiting indefinitely. Techniques like priority aging help.
 - **Deadlocks**: Improper resource allocation (e.g., in device queues) can cause processes to block each other.

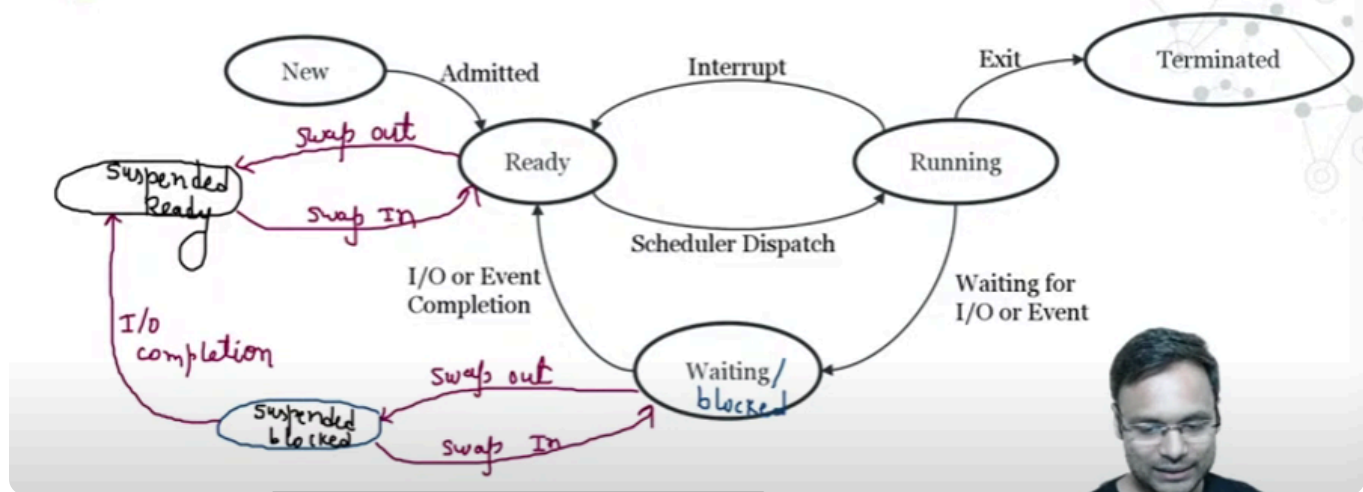


(Reference: Scheduling Queues)



(Reference: Queue Flow)

Updated Process States



(Reference: 7-State Process Diagram)

This covers CPU scheduling comprehensively, from basic concepts (what it does) to advanced details (algorithms, trade-offs, and multiprogramming). Let me know if you need further clarification or additional topics!